

Practical Week - 5

library - fastapi - project

1. FastAPI

It is a python framework used to build fast REST APIs.

```
> from fastapi import FastAPI  
app = FastAPI()
```

2. API Endpoint

An endpoint is a URL where a client can send a request to perform an operation.

```
> @app.get("/students")  
def get_students():  
    return {"message": "Students list"}
```

3. HTTP methods

GET : retrieve the data

POST : create data

PUT : update data

DELETE : delete data

```
Ex: @app.post("/students")  
def add_student():  
    return {"message": "Student added"}
```

4. Pydantic Model

A Pydantic Model defines the structure and data types of request data and validates it.

```
> from pydantic import BaseModel
```

```
class Student(BaseModel):
```

```
    name: str
```

```
    age: int
```

5. Field Validation

Field() adds restrictions to Pydantic fields.

```
> from pydantic import Field
```

```
age: int = Field(age=18)
```

Here, $ge=18$ means age must be 18 or greater.

6. SQLAlchemy

SQLAlchemy is a Python library used to communicate with databases using Python objects and SQL operations.

Ex:

```
student = StudentModel(name="Priyanka",  
                        age=20)
```

7. MySQL Connection

Database connection allows the FastAPI application to communicate with MySQL.

FastAPI $\rightarrow$ SQLAlchemy $\rightarrow$ MySQL

8. Async Programming

Asynchronous programming allows other tasks to run while waiting for operations such as database queries.

```
> async def get_data():  
    result = await session.execute(query)
```

- `async` → defines an asynchronous function

- `await` → waits for an async operation.

9. Dependency Injection

Dependency Injection allows FastAPI to automatically provide required resources such as database session.

```
> def get_db():  
    ...
```

```
@app.get("/students")
```

```
def get_students(db = Depends(get_db)):  
    ...
```

10. Background Tasks

Background Tasks runs a task after sending the API response.

```
> from fastapi import BackgroundTasks
```

```
@app.post("/send")
```

```
def send(background_tasks: BackgroundTasks):  
    background_tasks.add_task(my_task)  
    return {"message": "Done"}
```

11. Password Hashing

It converts a password into a secure hash instead of storing the original password.

> hashed = pwd_context.hash("12345")

Password verification:

```
pwd_context.verify("12345", hashed)
```

12. JWT Authentication

JWT is a token used to authenticate users & protect API endpoints.

Basic Flow:

Logic → JWT Token → Send token → Verify token

↓

Access protected API

Example token creation:

```
token = jwt.encode({"sub": "101"}, SECRET_KEY,
```

```
algorithm="HS256")
```


3. HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
422	Validation Error

4. Embeddings

An embedding represents text/data as a numerical vector so that similar content can be compared.

Ex: "apple" $\rightarrow$ [0.21, 0.45, 0.78, ...]

5. Cosine Similarity

It measures how similar two vectors are:

- Value close to 1 $\rightarrow$ more similar
- Value close to 0 $\rightarrow$ less similar

6. Swagger UI

It provides an interface to test FastAPI endpoints.

Usually available at:

/docs